Lab 7: Fuzzing for Input Validation Bugs

Name	Gnanasagar H
SRN	PES1UG24CS171
Section	5 - 'C'
Assigned repo	SETAPESU26/L6_L7_43_warranty_claims

Objective

The aim was to fuzz the Lab 7 script from my assigned repo with Hypothesis, find the inputs that break it, and then fix the code so it handles them. The module, warranty_claims.py, has three functions. parse_claim reads a plan and an age in months written as *plan:months*. age_average takes product ages as a comma separated list and returns the average. payable_charge reads *charge|discount*, adds 18% tax to the charge, subtracts the discount and never goes below zero.

Step 1: Installing dependencies

I installed both libraries with `pip install hypothesis` and `pip install pytest`. The versions used were Python 3.13, pytest 9.1.1 and Hypothesis 6.169.1.

Step 2: Running the buggy script

I ran `python warranty_claims.py` four times. With the sample inputs from the prompts it works. Leaving out the colon in the plan input, putting an empty value in the age list, or leaving out the pipe in the charge input each crashes the program with a ValueError and a traceback.

```
PS C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims> python warranty_claims.py
==== Running parse_claim ====
Enter plan:age in months (e.g. standard:30): standard:30
Plan and Age: ('Standard', 30.0)

==== Running age_average ====
Enter product ages as CSV (e.g. 12,24,36): 12,24,36
Average Age: 24

==== Running payable_charge ====
Enter charge|discount (e.g. 1500|200): 1500|200
Payable Charge: 1570.0

PS C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims> python warranty_claims.py
==== Running parse_claim ====
Enter plan:age in months (e.g. standard:30): standard
Traceback (most recent call last):
  File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\warranty_claims.py", line 31, in <module>
    main()
    ~~~~^^
  File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\warranty_claims.py", line 17, in main
    result = parse_claim(user_input)
  File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\warranty_claims.py", line 2, in parse_claim
    plan, months = text.split(':')
    ^^^^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)

PS C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims> python warranty_claims.py
==== Running parse_claim ====
Enter plan:age in months (e.g. standard:30): premium:18
Plan and Age: ('Premium', 18.0)

==== Running age_average ====
Enter product ages as CSV (e.g. 12,24,36): 12,,36
Traceback (most recent call last):
  File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\warranty_claims.py", line 31, in <module>
    main()
    ~~~~^^
  File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\warranty_claims.py", line 22, in main
    result = age_average(user_input)
  File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\warranty_claims.py", line 6, in age_average
    values = [int(r) for r in ages_csv.split(',')]
              ~~~^^^
ValueError: invalid literal for int() with base 10: ''

PS C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims> python warranty_claims.py
==== Running parse_claim ====
Enter plan:age in months (e.g. standard:30): basic:6
Plan and Age: ('Basic', 6.0)

==== Running age_average ====
Enter product ages as CSV (e.g. 12,24,36): 10
Average Age: 10

==== Running payable_charge ====
Enter charge|discount (e.g. 1500|200): 1500
Traceback (most recent call last):
  File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\warranty_claims.py", line 31, in <module>
    main()
    ~~~~^^
  File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\warranty_claims.py", line 27, in main
    result = payable_charge(user_input)
  File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\warranty_claims.py", line 10, in payable_charge
    charge, discount = text.split('|')
    ^^^^^^^^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)
```

Figure 1: Output of the original script

Step 3: Writing the Hypothesis tests

The starter test file only had three empty functions. I filled them in and added a few more, and saved the result as test_processor.py. There are two kinds of tests in it.

The first three tests feed each function any Unicode text or None, with 500 examples per test, and fail if the function raises anything. The other six check behaviour. Valid inputs built from strategies have to give sensible answers: the plan name is title-cased, the average sits between the smallest and largest age, and the payable charge is finite and never negative. Inputs that should be rejected, such as nan, inf, negative numbers and a trailing comma in the age list, have to come back as None.

```
import math

from hypothesis import given, settings, strategies as st
from warranty_claims import parse_claim, age_average, payable_charge

plans = st.sampled_from(["basic", "standard", "premium", " Standard ", "PREMIUM"])
numbers = st.floats(min_value=0, max_value=1e6, allow_nan=False, allow_infinity=False)
ages = st.integers(min_value=0, max_value=600)


@settings(max_examples=500)
@given(st.text() | st.none())
def test_parse_claim_no_crash(s):
    try:
        parse_claim(s)
    except Exception as e:
        assert False, f"parse_claim crashed with: {e!r}"


@settings(max_examples=500)
@given(st.text() | st.none())
def test_age_average_no_crash(s):
    try:
        age_average(s)
    except Exception as e:
        assert False, f"age_average crashed with: {e!r}"


@settings(max_examples=500)
@given(st.text() | st.none())
def test_payable_charge_no_crash(s):
    try:
        payable_charge(s)
    except Exception as e:
        assert False, f"payable_charge crashed with: {e!r}"


@given(plans, numbers)
def test_parse_claim_valid(plan, months):
    result = parse_claim(f"{plan}:{months}")
    assert result == (plan.strip().title(), months)


@given(st.from_regex(r"\A[a-z]*:(nan|inf|-inf|-[1-9][0-9]*)\Z"))
def test_parse_claim_rejects_bad_months(s):
    assert parse_claim(s) is None


@given(st.lists(ages, min_size=1, max_size=20))
def test_age_average_valid(values):
    result = age_average(",".join(str(v) for v in values))
    assert result is not None
    assert min(values) <= result <= max(values)
```

```
@given(st.lists(ages, max_size=5))
def test_age_average_empty_item(values):
    assert age_average(",".join(str(v) for v in values) + ",") is None


@given(numbers, numbers)
def test_payable_charge_valid(charge, discount):
    result = payable_charge(f"{charge}|{discount}")
    assert result is not None
    assert result >= 0
    assert math.isfinite(result)


@given(st.sampled_from(["nan", "inf", "-inf", "-5"]), numbers)
def test_payable_charge_rejects_bad_charge(charge, discount):
    assert payable_charge(f"{charge}|{discount}") is None
```

Figure 2: test_processor.py

Step 4: Running the tests on the buggy script

Running `python -m pytest test_processor.py` gave 6 failures and 3 passes. The output was too long for one screen, so it is split into four parts below. Hypothesis shrank each failure down to a small input, and these are listed in the table.

Function	Smallest failing input	What happened
parse_claim	None	AttributeError on None.split
parse_claim	'' (empty string)	ValueError, no colon to unpack
parse_claim	':'	ValueError, float('') fails
parse_claim	':nan'	Accepted, returned ('', nan)
age_average	None	AttributeError on None.split
age_average	'' and '12,'	ValueError, int('') fails
payable_charge	None	AttributeError on None.split
payable_charge	''	ValueError, no pipe to unpack
payable_charge	'|'	ValueError, float('') fails
payable_charge	'nan|0.0'	Returned nan as the charge

The nan cases are the interesting ones because nothing crashes. max(nan, 0) returns nan, so the charge comes out as nan and nobody gets an error. A normal unit test would not have tried that value. Writing the rule "the result must be a real number that is not negative" was enough for Hypothesis to find it.

```
PS C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims> python -m pytest test_processor.py
============================= test session starts =============================
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims
plugins: anyio-4.15.1, hypothesis-6.169.1, platformdirs-4.13.7
collected 9 items

test_processor.py FFF.F.F.F                                              [100%]

================================== FAILURES ===================================
_____________________________ test_parse_claim_no_crash _____________________________
  + Exception Group Traceback (most recent call last):
  |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\test_processor.py", line 12, in
test_parse_claim_no_crash
  |     @given(st.text() | st.none())
  |                      ^^^
  |   File "C:\Users\Temp\AppData\Local\Programs\Python\Python313\Lib\site-packages\hypothesis\core.py", line 2294, in
wrapped_test
  |     raise the_error_hypothesis_found
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\test_processor.py", line 15, in
test_parse_claim_no_crash
    |     parse_claim(s)
    |     ~~~~~~~~~~~~^^^
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\warranty_claims.py", line 3, in parse_claim
    |     return plan.strip().title(), float(months)
    |                                  ~~~~~^^^^^^^^
    | ValueError: could not convert string to float: ''
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\test_processor.py", line 17, in
test_parse_claim_no_crash
    |     assert False, f"parse_claim crashed with: {e!r}"
    | AssertionError: parse_claim crashed with: ValueError("could not convert string to float: ''")
    | assert False
    | Failing test case: test_parse_claim_no_crash(
    |     s=':',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\test_processor.py", line 15, in
test_parse_claim_no_crash
    |     parse_claim(s)
    |     ~~~~~~~~~~~~^^^
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\warranty_claims.py", line 2, in parse_claim
    |     plan, months = text.split(':')
    |     ^^^^^^^^^^^^
    | ValueError: not enough values to unpack (expected 2, got 1)
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\test_processor.py", line 17, in
test_parse_claim_no_crash
    |     assert False, f"parse_claim crashed with: {e!r}"
    | AssertionError: parse_claim crashed with: ValueError('not enough values to unpack (expected 2, got 1)')
    | assert False
    | Failing test case: test_parse_claim_no_crash(
    |     s='',  # or any other generated value
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\test_processor.py", line 15, in
test_parse_claim_no_crash
    |     parse_claim(s)
    |     ~~~~~~~~~~~~^^^
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\warranty_claims.py", line 2, in parse_claim
    |     plan, months = text.split(':')
    |                    ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\test_processor.py", line 17, in
test_parse_claim_no_crash
    |     assert False, f"parse_claim crashed with: {e!r}"
    | AssertionError: parse_claim crashed with: AttributeError("'NoneType' object has no attribute 'split'")
    | assert False
    | Failing test case: test_parse_claim_no_crash(
    |     s=None,  # or any other generated value
```

Figure 3: Failing test run, part 1 of 4



Figure 4: Failing test run, part 2 of 4

```
    |
    | Traceback (most recent call last):
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\test_processor.py", line 35, in
test_payable_charge_no_crash
    |     assert False, f"payable_charge crashed with: {e!r}"
    | AssertionError: payable_charge crashed with: ValueError("could not convert string to float: ''")
    | assert False
    | Failing test case: test_payable_charge_no_crash(
    |     s='|',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\test_processor.py", line 33, in
test_payable_charge_no_crash
    |     payable_charge(s)
    |     ~~~~~~~~~~~~~~~^^^
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\warranty_claims.py", line 10, in payable_charge
    |     charge, discount = text.split('|')
    |     ^^^^^^^^^^^^^^^^
    | ValueError: not enough values to unpack (expected 2, got 1)
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\test_processor.py", line 35, in
test_payable_charge_no_crash
    |     assert False, f"payable_charge crashed with: {e!r}"
    | AssertionError: payable_charge crashed with: ValueError('not enough values to unpack (expected 2, got 1)')
    | assert False
    | Failing test case: test_payable_charge_no_crash(
    |     s='',  # or any other generated value
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\test_processor.py", line 33, in
test_payable_charge_no_crash
    |     payable_charge(s)
    |     ~~~~~~~~~~~~~~~^^^
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\warranty_claims.py", line 10, in payable_charge
    |     charge, discount = text.split('|')
    |                        ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
    |
    | During handling of the above exception, another exception occurred:
    |
    | Traceback (most recent call last):
    |   File "C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims\test_processor.py", line 35, in
test_payable_charge_no_crash
    |     assert False, f"payable_charge crashed with: {e!r}"
    | AssertionError: payable_charge crashed with: AttributeError("'NoneType' object has no attribute 'split'")
    | assert False
    | Failing test case: test_payable_charge_no_crash(
    |     s=None,  # or any other generated value
    | )
    +-----------------------------------
_____________________ test_parse_claim_rejects_bad_months _____________________
test_processor.py:45: in test_parse_claim_rejects_bad_months
    def test_parse_claim_rejects_bad_months(s):
                   ^^^
test_processor.py:46: in test_parse_claim_rejects_bad_months
    assert parse_claim(s) is None
E   AssertionError: assert ('', nan) is None
E    +  where ('', nan) = parse_claim(':nan')
E   Failing test case: test_parse_claim_rejects_bad_months(
E       s=':nan',  # or any other generated value
E   )
_____________________________ test_age_average_empty_item ______________________________
test_processor.py:57: in test_age_average_empty_item
    def test_age_average_empty_item(values):
                   ^^^
test_processor.py:58: in test_age_average_empty_item
    assert age_average(",".join(str(v) for v in values) + ",") is None
           ^^^^^^^^^^^^^^^^
```

```
    assert payable_charge(f"{charge}|{discount}") is None
E   AssertionError: assert nan is None
E    +  where nan = payable_charge('nan|0.0')
E   Failing test case: test_payable_charge_rejects_bad_charge(
E       # The test always failed when commented parts were varied together.
E       charge='nan',  # or any other generated value
E       discount=0.0,  # or any other generated value
E   )
=========================== short test summary info ============================
FAILED test_processor.py::test_parse_claim_no_crash - ExceptionGroup: Hypothe...
FAILED test_processor.py::test_age_average_no_crash - ExceptionGroup: Hypothe...
FAILED test_processor.py::test_payable_charge_no_crash - ExceptionGroup: Hypo...
FAILED test_processor.py::test_parse_claim_rejects_bad_months - AssertionErro...
FAILED test_processor.py::test_age_average_empty_item - ValueError: invalid l...
FAILED test_processor.py::test_payable_charge_rejects_bad_charge - AssertionE...
======================== 6 failed, 3 passed in 3.75s ========================
```

Figure 6: Failing test run, part 4 of 4

Step 5: Fixing the script

Here is what I changed in warranty_claims.py:

- Every function first checks that it was given a string, and returns None if it was not.
- parse_claim and payable_charge split the input and check that there are exactly two parts. Before, they unpacked straight away, and that crashed when the separator was missing or appeared twice.
- A small helper, _to_number, does the float conversion inside a try block and turns down nan, inf and negative values. Months, charge and discount all go through it.
- parse_claim also turns down an empty plan name.
- age_average checks each item and only accepts plain ASCII digits, so empty items, signs, decimals and non-ASCII digits are all rejected. It now returns the real average rounded to two decimals, where the old version used floor division.
- payable_charge rounds the result to two decimals and checks that the total is finite.
- main() prints a short message about the expected format when a function returns None, so bad input no longer ends in a traceback.

I ran the fixed script with good input and then with the inputs that crashed it before. Each bad input now gets a message and the program carries on.

```
PS C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims> python warranty_claims.py
==== Running parse_claim ====
Enter plan:age in months (e.g. standard:30): standard:30
Plan and Age: ('Standard', 30.0)

==== Running age_average ====
Enter product ages as CSV (e.g. 12,24,36): 12,24,36
Average Age: 24.0

==== Running payable_charge ====
Enter charge|discount (e.g. 1500|200): 1500|200
Payable Charge: 1570.0

PS C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims> python warranty_claims.py
==== Running parse_claim ====
Enter plan:age in months (e.g. standard:30): standard
Invalid input. Expected format: plan:months

==== Running age_average ====
Enter product ages as CSV (e.g. 12,24,36): 12,,36
Invalid input. Expected format: comma separated whole numbers

==== Running payable_charge ====
Enter charge|discount (e.g. 1500|200): nan|200
Invalid input. Expected format: charge|discount

PS C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims> python warranty_claims.py
==== Running parse_claim ====
Enter plan:age in months (e.g. standard:30): premium:-4
Invalid input. Expected format: plan:months

==== Running age_average ====
Enter product ages as CSV (e.g. 12,24,36):
Invalid input. Expected format: comma separated whole numbers

==== Running payable_charge ====
Enter charge|discount (e.g. 1500|200): 1500
Invalid input. Expected format: charge|discount
```

Figure 7: Output of the fixed script

Step 6: Running the tests on the fixed script

With the fixed script in place, all 9 tests pass, including the three no-crash tests that try 500 random inputs each.

```
PS C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims> python -m pytest test_processor.py
============================= test session starts =============================
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Temp\AppData\Local\Programs\Python\Python313\python.exe
hypothesis profile 'default'
rootdir: C:\Users\Temp\Desktop\SE_Experiential_Learning\Lab-7\warranty_claims
plugins: anyio-4.15.1, hypothesis-6.169.1, platformdirs-4.13.7
collecting ... collected 9 items

test_processor.py::test_parse_claim_no_crash PASSED                      [ 11%]
test_processor.py::test_age_average_no_crash PASSED                      [ 22%]
test_processor.py::test_payable_charge_no_crash PASSED                   [ 33%]
test_processor.py::test_parse_claim_valid PASSED                         [ 44%]
test_processor.py::test_parse_claim_rejects_bad_months PASSED            [ 55%]
test_processor.py::test_age_average_valid PASSED                         [ 66%]
test_processor.py::test_age_average_empty_item PASSED                    [ 77%]
test_processor.py::test_payable_charge_valid PASSED                      [ 88%]
test_processor.py::test_payable_charge_rejects_bad_charge PASSED         [100%]

============================== 9 passed in 1.12s ==============================
```

Figure 8: All tests passing on the fixed script

Conclusion

The original code assumed every input would follow the format in the prompt. Within a few seconds Hypothesis found ten failing inputs across the three functions. Most were plain crashes, and two were nan values that slipped through quietly. What made the fix work was checking the input before using it rather than catching errors afterwards: confirm the type, count the parts, and only accept numbers that are finite and not negative. Writing tests as rules that should hold for any input caught cases I would not have thought to write by hand.